

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: OPERATING SYSTEMS    COURSE CODE: CSA04**

---

### **COURSE OUTCOME COVERED**

**CO2:** Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

*Assessment Tool Weightage: 30%*

---

Dr. M. Anoop

**QUESTIONS: 10**

**Total Marks:50**

Annexure A

**Scenario based**

---

### ***Code of Conduct:***

*I Atchaya Chandran R(192521394) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

*R. Atchay*

## Question 1: CPU Scheduling in a Real-Time Ticket Booking System (10 Marks)

### 1. Suitable Scheduling Algorithm

The most suitable scheduling method for this scenario is **Preemptive Priority Scheduling with Aging** (or a **Multilevel Feedback Queue - MLFQ**).

### 2. Justification

- **High Priority Execution:** Payment gateway transactions () are time-critical financial operations that must execute immediately to avoid transaction timeouts.
- **Balanced Response:** Seat checks () receive fast processing as medium-priority tasks once high-priority payment tasks finish.
- **Prevention of Delay:** Standard priority scheduling would cause low-priority tasks ( - Ticket confirmation and printing) to freeze during peak festival load. Incorporating **Aging** incrementally increases the priority of processes as they wait, guaranteeing that ticket confirmations are completed without excessive waiting times.

### 3. Sample Execution Order

Assuming processes arrive concurrently or in rapid succession:

1. **(Payment Gateway)** executes first due to highest priority.
2. **(Seat Availability Check)** executes next.
3. **(Ticket Confirmation)** executes once higher-priority queues are served or when its priority ages up.

If a new payment request () arrives while or is running, the CPU immediately preempts / to serve .

### 4. Drawback in Real-Time Systems

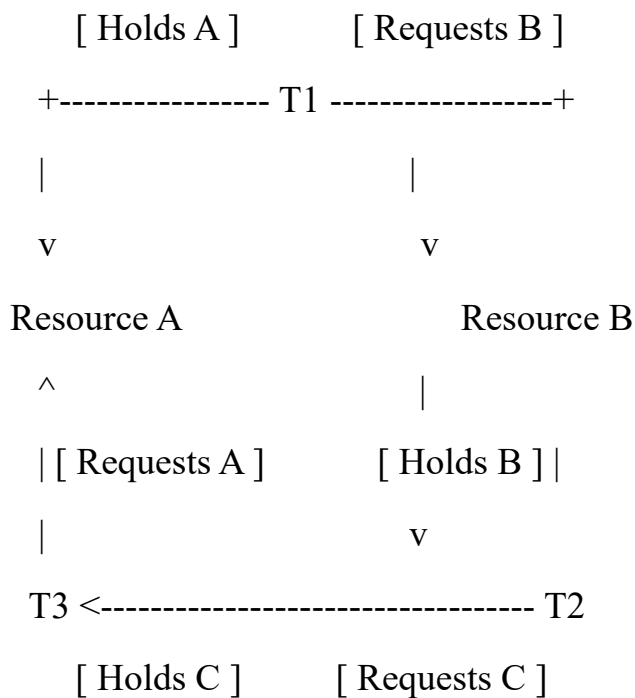
- **Starvation / Indefinite Blocking:** In pure priority scheduling without aging, if high-priority payment requests () continually arrive during peak festival hours, low-priority tasks () will never get CPU time, leading to ticket confirmation delays or dropped sessions.

## Question 2: Deadlock in a Banking Transaction Server (10 Marks)

### 1. Text-Based Resource Allocation Graph (RAG)

- **Nodes:**
  - Processes:
  - Resources (Single-instance databases):
- **Allocated Edges (Resource Process):**
  - ( holds Account DB )
  - ( holds Customer Info DB )
  - ( holds Transaction Logs )
- **Request Edges (Process Resource):**
  - ( requests DB )
  - ( requests DB )
  - ( requests DB )

Plaintext



## 2. Deadlock Identification

**Yes, a deadlock has occurred.**

**Justification:** All four Coffman conditions are present simultaneously:

1. **Mutual Exclusion:** Databases and are held exclusively by individual transactions.
2. **Hold and Wait:** holds while requesting ; holds while requesting ; holds while requesting .
3. **No Preemption:** Database locks cannot be forcibly confiscated from active transactions.
4. **Circular Wait:** A closed dependency loop exists in the single-instance resource allocation graph:

## 3. Recommended Technique: Deadlock Prevention via Resource Ordering

Enforce a strict **Global Lock Hierarchy** across all server transactions. Assign a fixed numerical order to resources (). Require every transaction to request database locks strictly in increasing numerical order. Because no transaction can request a lower-numbered resource while holding a higher-numbered one, circular wait becomes mathematically impossible.

## 4. Redesigning Lock Order

Establish the standard locking order: .

- **(Money Transfer):** Acquires , then acquires .
- **(Passbook Update):** Acquires , then acquires .
- **(Monthly Statement):** Redesigned to acquire first, then acquire (instead of holding and waiting for ).

## Question 3: Mobile OS App Switching Lag (10 Marks)

### 1. Reason for High CPU Idle Time

Mobile applications (like WhatsApp, social media, media players) are primarily **I/O-bound**—they spend significant time waiting for network responses, touchscreen inputs, or storage reads. When waiting for I/O operations to complete, processes move from the Running state to the **Blocked/Waiting state**. If all 20 background apps are simultaneously waiting on network or storage I/O, the ready queue becomes empty, leaving the CPU idle despite many active apps.

## 2. Evaluation of Time Quantum ()

The current quantum size **hurts performance**.

- A quantum is too long for user interface tasks, causing noticeable lag and delayed response during app switching.
- Applying Round Robin uniformly across 20 background apps forces the OS to waste context switching overhead on non-interactive background tasks.

## 3. Recommendation

Implement a **Multilevel Feedback Queue (MLFQ)** scheduler:

- **Foreground / Interactive Apps (WhatsApp UI, Media Controls):** Assigned to high-priority queues with a small time quantum () for fast user interaction.
- **Background Apps:** Assigned to lower-priority queues with larger quanta (), executing only when no foreground tasks are waiting.

## 4. Impact on Switching Lag and Throughput

- **Reduced Lag:** User interactions and foreground application switches immediately preempt background tasks, eliminating UI stutter.
- **Improved Throughput:** The CPU executes ready, actionable tasks rather than wasting cycles cycling through processes waiting on network or disk I/O.

## Question 4: Deadlock Risk in an Operating System Update (10 Marks)

### 1. Deadlock Risk Identification

**Yes, this sequence leads to a deadlock.**

### 2. Exact Circular Wait Condition

Modules `and` form a circular dependency chain over resources `and` :

- holds (Disk I/O) and waits for (Kernel lock).
- holds (Kernel lock) and waits for (Configuration lock).
- holds (Configuration lock) and waits for (Disk I/O).

(Note: Module `operates` independently on CPU validation tasks and is not part of the deadlock cycle).

### 3. Banker's Algorithm Style Allocation Strategy

Before granting any resource request during update execution:

1. The OS maintains state vectors: `,` `,` `,` and `.`

2. When a module (e.g., ) requests an additional resource (), the OS runs a **Safety Algorithm** simulation.
3. The system checks if granting allows at least one execution sequence (e.g., ) where all modules can obtain their max demands and complete.
4. If granting the request puts the system into an **unsafe state**, is forced to wait until other modules release resources.

#### 4. Module Prioritization Strategy for High CPU Utilization

- **Co-Scheduling Complementary Modules:** Pair I/O-intensive modules (: Copy files) with CPU-intensive modules (: Validate integrity) so CPU and disk controllers are utilized simultaneously.
- **Lock Allocation Hierarchy:** Standardize the acquisition order () across all update modules to eliminate hold-and-wait cycles during installation.

### Question 5: Cloud Server CPU Utilization Analysis (10 Marks)

#### 1. Calculation of CPU Utilization

Given parameters:

- (Probability of I/O wait)
- (Number of Virtual Machines)

Formula:

Step-by-step evaluation of :

Calculate CPU Utilization:

#### 2. Server Load Interpretation

The server is **optimally loaded (~79.03% utilization)**. Although each individual VM spends of its time waiting on I/O, maintaining concurrent VMs keeps the host CPU active nearly of the total operating time.

#### 3. Impact of Changing the Number of VMs ()

- **Increasing VMs ():**

- *Effect:* Increases CPU utilization and transaction throughput. However, pushing too high risks RAM saturation and severe I/O bus contention.

- **Decreasing VMs ():**

- *Effect:* Results in server underutilization, increasing CPU idle time to and lowering overall cloud platform throughput.

#### 4. Two OS-Level Strategies to Improve CPU Utilization

1. **Asynchronous Non-blocking I/O Subsystems:** Utilize kernel-level asynchronous I/O frameworks (e.g., `io_uring`) so VM processes do not enter a blocked wait state during network and storage operations, freeing CPU cycles for other active processes.
2. **Workload-Aware VM Co-Scheduling:** Implement hypervisor-level schedulers that automatically pair compute-bound VMs with I/O-bound VMs on shared host CPU cores to balance resource usage and keep CPU core occupancy high.

#### Question 6: Autonomous Vehicle Sensor Pipeline Scheduling (10 Marks)

##### Scenario

An autonomous vehicle OS manages three sensor processing threads:

- : Emergency Collision Avoidance (High priority, strict real-time deadlines)
- : Camera/LiDAR Perception & Object Detection (Medium priority, high compute demand)
- : Route Planning & Telemetry Logging (Low priority, background tasks)

During high-density traffic, engineers note that completely stops updating maps, while occasionally drops object-detection frames when triggers continuous monitoring.

##### Tasks

1. **Identify the most suitable scheduling algorithm (FCFS, SJF, Preemptive Priority with Multilevel Feedback Queues).**
2. **Justify why your chosen method improves CPU utilization and ensures safety.**
3. **Provide a sample execution order for the above categories.**
4. **Discuss one drawback of this scheduling method in real-time safety systems.**

##### Step-by-Step Solution

##### 1. Scheduling Algorithm Identification

The optimal scheduling strategy is **Preemptive Fixed-Priority Scheduling with Hard Real-Time Deadlines and Priority Inheritance / Aging**.

##### 2. Justification

- **Safety First:** Emergency braking () must immediately preempt any executing task ( or ) to prevent accidents.
- **Predictable Latency:** Fixed-priority preemption guarantees lower bounds on reaction times for critical safety events.

- Prevention of Starvation:** Combining priority scheduling with **Aging** or a **Multilevel Queue** ensures low-priority processes () eventually receive CPU time during long periods of clear driving.

### 3. Execution Sequence

Assuming concurrent arrival:

1. **(Emergency Avoidance)** runs immediately to completion or until its real-time burst completes.
2. **(Object Detection)** executes during active driving cycles.
3. **(Telemetry/Maps)** executes during CPU idle gaps or when boosted via aging.

#### 4. Drawback in Real-Time Systems

- **Priority Inversion / Overhead:** Priority scheduling in real-time systems can cause **Priority Inversion** (where a high-priority thread waits on a shared resource held by a lower-priority thread). Resolving this requires complex protocol support (e.g., Priority Inheritance Protocol), which increases OS kernel overhead.

### Question 7: Deadlock in a Distributed Database Cluster (10 Marks)

## Scenario

A distributed financial server processes transactions across three independent database nodes: Node A (User Accounts), Node B (Order Queue), and Node C (Audit Vault).

- Transaction holds Node A and requests Node B.
- Transaction holds Node B and requests Node C.
- Transaction holds Node C and requests Node A.

## Tasks

1. **Draw the resource request situation (text-based RAG).**
2. **Identify whether a deadlock has occurred and justify your answer.**
3. **Recommend one specific technique (avoidance/prevention/recovery) and explain how it fixes the issue.**
4. **Suggest a redesign for the locking order to avoid future deadlocks.**

### Step-by-Step Solution

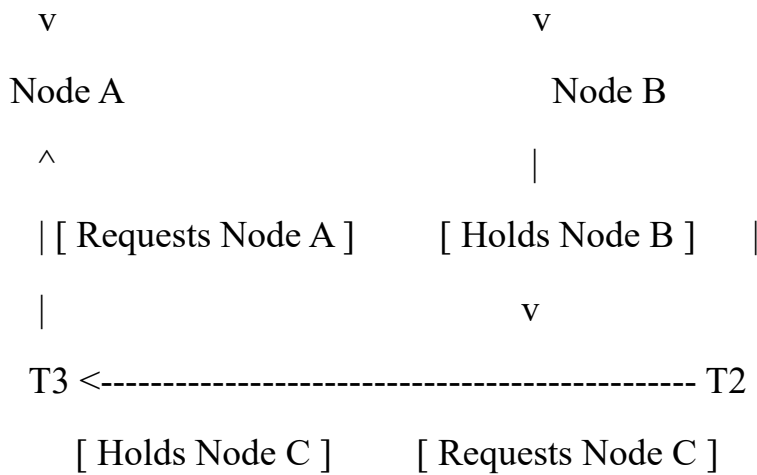
## 1. Text-Based Resource Allocation Graph (RAG)

- **Processes:**
- **Resources (Nodes):**

Plaintext

[ Holds Node A ]      [ Requests Node B ]

$$+ \text{-----} T1 \text{-----} +$$



## 2. Deadlock Identification

**Yes, a deadlock has occurred.**

**Justification:** The system satisfies all four Coffman conditions:

1. **Mutual Exclusion:** Nodes and enforce exclusive lock access.
2. **Hold and Wait:** holds while requesting ; holds while requesting ; holds while requesting .
3. **No Preemption:** Distributed locks cannot be forcibly released without risking database corruption.
4. **Circular Wait:** A closed dependency loop exists:

## 3. Recommended Technique: Deadlock Prevention via Resource Hierarchy

Assign a strict global rank to all database nodes: . Enforce a rule that transactions must acquire locks strictly in increasing rank order.

## 4. Redesigned Locking Order

Enforce the sequence: .

- : Locks Node , then requests Node .
- : Locks Node , then requests Node .
- : Modified to request Node **before** requesting Node .

Because cannot lock Node without first acquiring Node , the circular chain is broken.

## Question 8: Video Encoding Engine CPU Quantum Optimization (10 Marks)

### Scenario

A high-throughput video processing server runs 3 parallel encoding threads (). The operating system uses Round Robin scheduling with a time quantum . Context switching overhead on this hardware is . Users complain that interactive playback stutters, and overall frame processing throughput is low.

### Tasks



1. **Analyze why paired with context switching overhead affects responsiveness and throughput.**
2. **Evaluate the performance impact if is reduced to vs. increased to .**
3. **Recommend a better scheduling method or quantum adjustment, with justification.**
4. **Explain how the change will improve responsiveness and system throughput.**

## Step-by-Step Solution

### 1. Performance Analysis

- **Throughput Loss:** At with , the system spends of its time purely on CPU context switches rather than encoding video frames.
- **Latency & Stutter:** A fixed time quantum introduces up to of latency for interactive UI threads waiting in the ready queue, causing visible audio/video frame drops.

### 2. Evaluation of Alternative Quanta

- **Reducing to :**
  - *Result:* CPU spends half its time switching threads, severely degrading throughput.
- **Increasing to :**
  - *Result:* CPU utilization and raw compute throughput improve, but interactive UI latency worsens, increasing frame playback stutter.

### 3. Recommendation

Implement a **Multilevel Feedback Queue (MLFQ)** scheduler:

- **Interactive/Audio Threads:** High-priority queue with for zero UI lag.
- **Batch Video Rendering Threads:** Low-priority queue with to maximize compute throughput.

### 4. System Impact

- **Eliminated Stutter:** User input and audio streams preempt render threads immediately.
- **Maximized Throughput:** Heavy render tasks execute with long uninterrupted execution windows (), dropping context switch overhead to under .

## Question 9: Multi-Resource Allocation in AI Model Pipelines (10 Marks)

### Scenario

An OS manages three hardware resources: GPU VRAM (), Host RAM (), and NVMe Swap (). Four AI pipeline modules are running:

- holds , waiting for
- holds , waiting for
- holds , waiting for
- requests only

### Tasks

1. **Identify whether this sequence leads to a deadlock, specifying the exact circular wait.**
2. **Propose Banker's Algorithm style resource allocation to evaluate safely serving .**
3. **Suggest a resource allocation ordering to prevent deadlocks.**
4. **Suggest how OS designers can prioritize modules to maximize CPU/GPU overlap.**

### Step-by-Step Solution

#### 1. Deadlock Potential & Circular Wait

Yes, a deadlock exists among and .

*(Module is blocked from acquiring because holds , but is not part of the primary cycle).*

#### 2. Banker's Algorithm Safety Evaluation

1. Before allocating to , the OS runs a **Safety Check Simulation**.
2. It constructs the vector .
3. The system verifies whether a safe sequence exists where all process demands can be met by available resources.
4. Because is fully allocated to and is stuck in an unsafe circular wait state, allocating to is denied, and is placed in a waiting state.

#### 3. Deadlock Prevention

Require all training modules to request resources in strict hierarchy (). No process can hold and request .

#### 4. Module Prioritization Strategy

- **I/O and Compute Pipelining:** Overlap CPU host data staging () with GPU tensor execution () using asynchronous double-buffering to maximize both CPU and GPU occupancy.

### Question 10: Multi-Tenant Container CPU Utilization (10 Marks)

#### Scenario

A cloud container server hosts containerized web services. Profile monitoring reveals that each container spends () of its operational time waiting on remote database network calls and storage I/O.

#### Tasks

1. **Calculate CPU utilization using .**
2. **Interpret whether the server is underutilized, optimal, or overloaded.**
3. **Predict how increasing the number of containers to or decreasing to affects utilization.**
4. **Propose two OS kernel-level strategies to improve CPU utilization.**

### Step-by-Step Solution

#### 1. Calculation of CPU Utilization

Given:

- 
- 

Evaluating :

Calculate final utilization:

## 2. Load Interpretation

The server is **optimally loaded (~89.99% utilization)**. Despite individual containers spending of their time waiting on network calls, running containers keeps the CPU core active for ~90% of uptime.

## 3. Impact of Changing Container Count ()

- **Increasing to :**
  - *Effect:* Increases CPU utilization near total capacity, but risks RAM thrashing and high context switching overhead.
- **Decreasing to :**
  - *Effect:* CPU idle time jumps to , resulting in severe hardware underutilization.

## 4. Two OS Kernel-Level Strategies

1. **Kernel Asynchronous I/O (io\_uring):** Replaces blocking I/O system calls with non-blocking ring buffers so thread execution does not stall while waiting for network responses.
2. **Workload-Aware Dynamic Thread Pooling:** Group compute-bound tasks and I/O-bound tasks onto shared CPU core affinity groups to keep CPU pipelines saturated without excessive context switching.